

Guide Complet de Préparation aux Entretiens Techniques

Java 8-21 • Spring Boot & WebFlux • Cloud & DevOps • Frontend

Support de révision technique

Résumé

Ce guide constitue un support complet de révision pour les entretiens techniques couvrant les technologies modernes du développement backend et frontend. Il combine concepts fondamentaux, évolutions majeures, bonnes pratiques et pièges courants à connaître pour réussir un entretien technique professionnel.

Table des matières

1	Java 8 à Java 21 : Évolution Complète et Concepts Avancés	5
1.1	Chronologie des versions et features majeures	5
1.2	Programmation Fonctionnelle : Lambdas et Streams	5
1.2.1	Comprendre les Lambdas	5
1.2.2	Streams : Traitement Déclaratif	6
1.2.3	Parallel Streams : Parallélisme et Pièges	6
1.3	Optional : Éliminer les NullPointerException	7
1.4	Interfaces Fonctionnelles et Références de Méthodes	7
1.4.1	Références de Méthodes	8
1.5	Records : Immutabilité et Données	8
1.6	Sealed Classes : Contrôle de l'Héritage	9
1.7	Pattern Matching Avancé (Java 21)	9
1.8	Virtual Threads (Java 21 - Project Loom)	10
1.9	Concepts Transversaux Importants	11
1.9.1	Immutabilité et Thread Safety	11
1.9.2	Composition vs Héritage	11
1.9.3	Principes SOLID	11
2	Spring Boot : Architecture Complète et Bonnes Pratiques	11
2.1	Architecture en Couches Classique	11
2.1.1	Pattern Service	12
2.1.2	DTOs et Conversion	12
2.2	Configuration Spring Boot	13
2.2.1	application.yml et Profils	13
2.2.2	ConfigurationProperties	14
2.3	Persistance avec Spring Data JPA	14
2.3.1	Repository Pattern	14
2.3.2	Relations et Lazy Loading	15
2.4	Gestion des Transactions	15
2.5	Internationalisation (i18n)	16
2.6	Gestion des Erreurs	17
2.7	Base de Données : Indexes et Schémas	18

2.7.1	Indexes pour Performances	18
2.7.2	Versioning des Schémas	18
3	Spring WebFlux : Programmation Réactive	19
3.1	Reactive Streams et Backpressure	19
3.2	Mono et Flux : Types de Streams	19
3.3	Opérateurs Réactifs Clés	19
3.4	Contrôle de Concurrency	20
3.5	WebFlux : Contrôleurs Réactifs	20
4	Messaging et Événements : Kafka	23
4.1	Architecture et Concepts Fondamentaux	23
4.2	Partitions et Parallélisme	23
4.3	Consumer Groups et Scalabilité	23
4.4	Offsets et Commits	24
4.5	Intégration Spring Kafka	24
4.6	Garanties et Sémantiques de Livraison	25
5	Cloud et Infrastructure : AWS Essentials	26
5.1	Compute Services	26
5.1.1	EC2 : Machines Virtuelles Élastiques	26
5.1.2	Containerization : Docker et ECS	26
5.1.3	ECR : Registry Docker Managé	26
5.2	Storage Services	26
5.2.1	S3 : Stockage Objet Illimité	26
5.3	Messaging : SQS et SNS	27
5.3.1	SQS : File de Messages Découplée	27
5.3.2	SNS : Publication/Souscription	27
5.4	CloudFront : CDN Global	27
6	CI/CD avec GitLab	28
6.1	Pipelines et Stages	28
6.2	Runners : Exécution Distribuée	28
6.3	Advanced Patterns	29
7	Frontend : Angular Moderne	30
7.1	Composants et Data Binding	30
7.2	RxJS et Observables	30
7.3	Pagination	31
7.3.1	Backend Pagination	31
7.3.2	Frontend Pagination	32
7.4	Guards et Authentification	32
7.5	Signals et Effects (Angular 17+)	33
7.6	Change Detection et Performance Angular	34
7.6.1	Change Detection Strategy	34
7.6.2	TrackBy avec *ngFor	34
7.7	Concurrency et Combinaison Observables	35
8	Concurrency et Programmation Distribuée	36
8.1	Modèles de Concurrency	36
8.1.1	Thread Safety et Visibilité	36
8.1.2	Deadlock et Solutions	36
8.1.3	Executor Framework	36

9	Patterns et Bonnes Pratiques Transversales	37
9.1	Gestion des Erreurs et Logging	37
9.1.1	Backend : Logging Structuré	37
9.1.2	Frontend : Gestion Centralisée	37
9.2	Testing	38
9.2.1	Backend : Tests Unitaires et Intégration	38
9.2.2	Frontend : Tests Angular	39
9.3	Stratégies de Cache et Invalidation	40
9.3.1	Backend Caching	40
9.3.2	Frontend Caching Strategy	40
9.4	Performance et Optimisation	41
9.4.1	Backend	41
9.4.2	Frontend	41
9.5	Debugging et Monitoring	41
9.5.1	Backend Monitoring	41
9.5.2	Frontend Debugging	42
9.6	Sécurité et Compliance AWS	43
9.6.1	IAM : Contrôle d'Accès Granulaire	43
9.6.2	VPC et Networking	43
9.6.3	Encryption et Secrets	43
9.7	Patterns AWS Avancés	43
9.7.1	Auto Scaling	43
9.7.2	Load Balancing	44
9.8	Cost Optimization AWS	44
10	Patterns d'Architecture Distribuée	44
10.1	CQRS et Event Sourcing	44
10.2	Saga Pattern pour Transactions Distribuées	44
10.3	Resilience Pattern	45
10.3.1	Circuit Breaker	45
11	Questions-Réponses : Préparation aux Entretiens	46
11.1	Questions Java et Programmation Fonctionnelle	46
11.1.1	Q1 : Quelle est la différence entre List et Stream en Java ?	46
11.1.2	Q2 : Pourquoi utiliser Optional plutôt que simplement vérifier null ?	46
11.1.3	Q3 : Expliquez la différence entre parallel() et Sequential() dans les Streams	46
11.1.4	Q4 : Qu'est-ce qu'une interface fonctionnelle et pourquoi c'est important ?	46
11.1.5	Q5 : Quels sont les avantages des Records par rapport aux classes ordinaires ?	46
11.1.6	Q6 : Expliquez comment Virtual Threads résout le problème de scalabilité	47
11.1.7	Q7 : Quel est l'impact du choix de collection sur les performances ?	47
11.1.8	Q8 : Qu'est-ce qu'une race condition et comment l'éviter ?	47
11.1.9	Q9 : Décrivez le cycle de vie des objets en Java	47
11.1.10	Q10 : Qu'est-ce que l'immuabilité et pourquoi c'est important en concurrence ?	47
11.2	Questions Spring Boot et Architecture Backend	47
11.2.1	Q1 : Expliquez l'architecture en couches et ses responsabilités	47
11.2.2	Q2 : Pourquoi utiliser DTOs au lieu de retourner directement des entités ?	48
11.2.3	Q3 : Quel est le problème N+1 et comment l'identifier/corriger ?	48
11.2.4	Q4 : Expliquez les niveaux d'isolations transactionnelles	48
11.2.5	Q5 : Comment gérer les exceptions efficacement en Spring Boot ?	48
11.2.6	Q6 : Quel est l'avantage de @Transactional et ses pièges ?	48
11.2.7	Q7 : Comment choisir entre XML et annotations pour configuration Spring ?	49

11.2.8	Q8 : Qu'est-ce que la propagation transactionnelle et ses modes ?	49
11.2.9	Q9 : Comment configurer et utiliser les profils Spring ?	49
11.2.10	Q10 : Expliquez comment Spring injecte les dépendances et ses cycles de vie	49
11.3	Questions Kafka, AWS et DevOps	49
11.3.1	Q1 : Expliquez le mécanisme de partitions dans Kafka et ses bénéfices	49
11.3.2	Q2 : Qu'est-ce qu'un Consumer Group et comment fonctionne la sémantique de livraison ?	49
11.3.3	Q3 : Quel est l'offset et comment Kafka track consommation ?	50
11.3.4	Q4 : Décrivez l'architecture AWS et services principaux	50
11.3.5	Q5 : Quel est différence entre S3, EBS et EFS ?	50
11.3.6	Q6 : Expliquez Auto Scaling et comment configurer	50
11.3.7	Q7 : Qu'est-ce qu'un IAM Role et pourquoi c'est important ?	50
11.3.8	Q8 : Comment optimiser coûts AWS ?	50
11.3.9	Q9 : Expliquez une CI/CD pipeline type	51
11.3.10	Q10 : Qu'est-ce qu'une Infrastructure as Code et pourquoi l'utiliser ?	51
11.4	Questions Frontend Angular et Patterns Avancés	51
11.4.1	Q1 : Expliquez la différence entre Observable et Promise	51
11.4.2	Q2 : Qu'est-ce que Change Detection et comment optimiser ?	51
11.4.3	Q3 : Expliquez RxJS switchMap vs mergeMap vs concatMap	51
11.4.4	Q4 : Comment gérer erreurs et retry en RxJS ?	52
11.4.5	Q5 : Quel est le cycle de vie des composants Angular ?	52
11.4.6	Q6 : Expliquez Guards (CanActivate, CanDeactivate)	52
11.4.7	Q7 : Comment optimiser performance Angular app ?	52
11.4.8	Q8 : Qu'est-ce que Dependency Injection et pourquoi c'est crucial ?	52
11.4.9	Q9 : Expliquez Signals et Effects (Angular 17+)	53
11.4.10	Q10 : Qu'est-ce que State Management et quand en avoir besoin ?	53

1 Java 8 à Java 21 : Évolution Complète et Concepts Avancés

1.1 Chronologie des versions et features majeures

L'évolution de Java représente une transformation progressive du langage vers la programmation fonctionnelle et réactive. **Java 8** marque un tournant décisif en 2014 avec l'introduction des lambdas et des streams, révolutionnant l'approche déclarative du traitement des données. Les versions ultérieures consolident cette direction tout en ajoutant des outils pour améliorer les performances et la lisibilité du code.

- **Java 8 (2014)** : Lambda expressions, Streams API, Optional, Interface fonctionnelles (@FunctionalInterface), Date/Time API (java.time), DefaultMethod dans les interfaces
- **Java 9 (2017)** : Module System (JPMS), JShell, Private methods dans les interfaces, try-with-resources amélioré
- **Java 10 (2018)** : Inférence de type local (var), Collections immuables, Garbage Collector expérimental (G1GC par défaut)
- **Java 11 (2018, LTS)** : HttpClient stable, String méthodes utilitaires (strip, lines), Opaque data classes preview
- **Java 14-16 : Features preview** : Records (immuables, avec générés equals/hashCode/toString), Pattern matching sur instanceof, Foreign Function Interface preview
- **Java 17 (2021, LTS)** : Sealed Classes (héritage restreint), Pattern matching stabilisé, FFI amélioré
- **Java 21 (2023, LTS)** : Virtual Threads (Project Loom pour concurrence légère), Pattern matching avancé, Record patterns, String templates preview

1.2 Programmation Fonctionnelle : Lambdas et Streams

1.2.1 Comprendre les Lambdas

Une lambda expression est une fonction anonyme qui peut être stockée dans une variable ou passée en argument. Syntaxe générale : (paramètres) -> { corps }. Les lambdas ne fonctionnent que pour les **interfaces fonctionnelles**, c'est-à-dire possédant une unique méthode abstraite.

```
// Avant Java 8 : verbeux
List<Integer> numbers = Arrays.asList(1, 2, 3, 4);
numbers.forEach(new Consumer<Integer>() {
    @Override
    public void accept(Integer n) {
        System.out.println(n);
    }
});

// Apr s Java 8 : concis
numbers.forEach(n -> System.out.println(n));

// Avec type explicite
numbers.forEach((Integer n) -> System.out.println(n));

// Corps multi-lignes
numbers.forEach(n -> {
    System.out.println("Nombre: " + n);
    System.out.println("Carr : " + (n * n));
});
```

1.2.2 Streams : Traitement Déclaratif

Les **streams** permettent une manipulation déclarative de collections. Caractéristiques clés :

- **Lazy evaluation** : les opérations intermédiaires (map, filter) ne s'exécutent que lorsqu'une opération terminale (collect, forEach, reduce) est appelée
- **Séquence immuable** : le stream ne modifie pas la source originale
- **Chainage fonctionnel** : composabilité des opérations
- **Pas d'index** : pas d'accès aléatoire contrairement aux listes

Exemple pratique d'un pipeline stream complet :

```
List<Personne> personnes = Arrays.asList(
    new Personne("Alice", 28),
    new Personne("Bob", 35),
    new Personne("Charlie", 22)
);

// Filtrer, transformer, réduire
String resultat = personnes.stream()
    .filter(p -> p.getAge() > 25)           // Filtre intermédiaire
    .map(Personne::getName)                 // Transformation
    .sorted()                              // Tri intermédiaire
    .collect(Collectors.joining(", "));     // Terminale

// Utilisation de collectors avancés
Map<Integer, List<String>> parAge = personnes.stream()
    .collect(Collectors.groupingBy(
        Personne::getAge,
        Collectors.mapping(Personne::getName, Collectors.toList())
    ));

// Statistiques
DoubleSummaryStatistics stats = personnes.stream()
    .collect(Collectors.summarizingDouble(Personne::getAge));
// stats.getAverage(), stats.getMin(), stats.getMax(), stats.getCount()
```

1.2.3 Parallel Streams : Parallélisme et Pièges

Les parallel streams utilisent un fork/join pool partagé pour traiter les données en parallèle. Cependant, leur utilisation requiert une grande vigilance :

```
// Peut être plus lent que séquentiel pour petites données
List<Integer> nombres = Arrays.asList(1, 2, 3, 4, 5);
nombres.parallelStream()
    .map(n -> n * n)
    .collect(Collectors.toList());

// Danger : modifications externes (race conditions)
List<String> resultat = new ArrayList<>();
personnes.parallelStream()
    .forEach(p -> resultat.add(p.getName())); // UNSAFE !

// Solution : utiliser collect au lieu de forEach
List<String> noms = personnes.parallelStream()
    .map(Personne::getName)
    .collect(Collectors.toList()); // SAFE
```

Bonnes pratiques : n'utiliser `parallelStream` que pour collections volumineux (>1000 éléments), opérations CPU-intensives, pas d'I/O bloquants, données stateless (pas d'accès partagé).

1.3 Optional : Éliminer les `NullPointerException`

Optional est un container qui représente une valeur optionnelle (présente ou absente). Il force le développeur à traiter explicitement le cas null, éliminant les `NullPointerException` inattendues.

```
// Cr ation
Optional<String> opt1 = Optional.of("valeur");           // Toujours non-
    vide
Optional<String> opt2 = Optional.ofNullable(null);       // Peut tre vide
Optional<String> opt3 = Optional.empty();               // Toujours vide

// Op rations de base
Optional<String> result = Optional.of("hello")
    .map(s -> s.toUpperCase())                          // "HELLO"
    .filter(s -> s.length() > 3)                       // Passe le filtre
    .map(s -> s.substring(0, 2));                       // "HE"

// Chainage avec flatMap
Optional<String> username = Optional.of("user123");
Optional<Integer> age = username
    .flatMap(user -> findUserById(user))                // Retourne
        Optional<User>
    .flatMap(user -> Optional.ofNullable(user.getAge()))
    .filter(a -> a >= 18);

// Extraire la valeur
String valeur = Optional.of("test")
    .orElse("d faut");                                  // "test"
String autre = Optional.empty()
    .orElse("d faut");                                  // "d faut"

Optional.of("action").ifPresent(System.out::println);
Optional.of("action").ifPresentOrElse(
    System.out::println,
    () -> System.out.println("Absent")
);
```

Anti-patterns courants : utiliser `Optional` en champ de classe ou paramètre de méthode est un anti-pattern. `Optional` est destiné aux valeurs de retour.

1.4 Interfaces Fonctionnelles et Références de Méthodes

Une **interface fonctionnelle** déclare exactement une méthode abstraite. L'annotation `@FunctionalInterface` (optionnelle mais recommandée) garantit cette propriété.

```
@FunctionalInterface
public interface Calcul {
    int effectuer(int a, int b);
}

// Impl ementation lambda
Calcul addition = (a, b) -> a + b;
Calcul soustraction = (a, b) -> a - b;
```

```
// Utilisation
int resultat = addition.effectuer(10, 5); // 15
```

Les interfaces fonctionnelles prédéfinies essentielles :

- Predicate<T> : prend T, retourne boolean — pour les filtres
- Function<T, R> : prend T, retourne R — transformations
- Consumer<T> : prend T, pas de retour — effets secondaires
- Supplier<T> : pas d'argument, retourne T — génération paresseuse
- BiFunction<T, U, R> : deux arguments, retourne R
- UnaryOperator<T> : T vers T
- BinaryOperator<T> : deux T vers T

1.4.1 Références de Méthodes

Les références de méthodes offrent une syntaxe compacte pour appeler une méthode existante via une lambda :

```
// 1. Référence de méthode statique
Function<String, Integer> parser = Integer::parseInt;
// équivalent : s -> Integer.parseInt(s)

// 2. Référence de méthode d'instance sur un objet fixe
String texte = "Hello World";
Consumer<Integer> afficher = System.out::println;
afficher.accept(42);

// 3. Référence de méthode d'instance sur un paramètre
BiFunction<String, String, Integer> comparaison = String::compareTo;
// str1.compareTo(str2)

// 4. Référence de constructeur
Function<String, Integer> creerInteger = Integer::new;
Integer valeur = creerInteger.apply("42");

// Cas réel : avec streams
List<String> mots = Arrays.asList("hello", "world");
List<Integer> longueurs = mots.stream()
    .map(String::length) // Référence String.length()
    .collect(Collectors.toList());
```

1.5 Records : Immutabilité et Données

Introduits en Java 14 et stabilisés en Java 16, les **records** sont des classes optimisées pour représenter des tuples de données immuables. Ils génèrent automatiquement :

- Le constructeur canonique
- Les accesseurs (au lieu de getters)
- equals(), hashCode(), toString()
- Immutabilité garantie (champs final)

```
// Record simple
public record Adresse(String rue, String ville, String codePostal) {}
```



```
// Utilisation
Adresse addr1 = new Adresse("123 Rue Principale", "Paris", "75001");
Adresse addr2 = new Adresse("123 Rue Principale", "Paris", "75001");
System.out.println(addr1.equals(addr2)); // true
System.out.println(addr1.rue());         // "123 Rue Principale"

// Record avec validation dans constructeur compact
public record Utilisateur(String pseudo, int age) {
    public Utilisateur {
        if (age < 0) throw new IllegalArgumentException("Age n gatif");
        if (pseudo == null) throw new NullPointerException();
    }
}

// Record avec m thode personnalis e
public record Produit(String nom, double prix) {
    public boolean estChER() {
        return prix > 100;
    }
}
```

1.6 Sealed Classes : Contrôle de l'Héritage

Les **sealed classes**, stabilisées en Java 17, restreignent quelles classes peuvent en hériter. Cela améliore la conception et rend le code plus prévisible pour pattern matching.

```
// D finition
public sealed class Figure permits Cercle, Rectangle, Triangle {
    abstract double aire();
}

public final class Cercle extends Figure {
    private double rayon;
    double aire() { return Math.PI * rayon * rayon; }
}

public final class Rectangle extends Figure {
    private double largeur, hauteur;
    double aire() { return largeur * hauteur; }
}

// Toute autre tentative d'h ritage provoque une erreur compile

// Avantage : exhaustivit avec pattern matching
String description = switch (figure) {
    case Cercle c -> "Cercle de rayon " + c.rayon;
    case Rectangle r -> "Rectangle " + r.largeur + "x" + r.hauteur;
    case Triangle t -> "Triangle";
};
```

1.7 Pattern Matching Avancé (Java 21)

Le pattern matching modernise les vérifications de type et structure de données :

```
// 1. Pattern matching sur type et destructuring (Java 16)
public String decrire(Object obj) {
    return switch (obj) {
```

```

        case String s when s.length() > 5 -> "Long string: " + s;
        case String s -> "String: " + s;
        case Integer i && i > 100 -> "Grand nombre: " + i;
        case Integer i -> "Nombre: " + i;
        case null -> "Null";
        default -> "Autre";
    };
}

// 2. Record patterns (Java 21)
record Point(int x, int y) {}
record Carre(Point coin1, Point coin2) {}

public void afficherCarre(Object obj) {
    if (obj instanceof Carre(Point(var x1, var y1), Point(var x2, var y2))) {
        System.out.println("Carré de " + x1 + "," + y1 + " et " + x2 + "," + y2);
    }
}

// 3. Array patterns (Java 21)
public boolean premier(int[] tab) {
    return switch(tab) {
        case [int head, int... tail] when head > 0 -> true;
        case [] -> false;
        default -> false;
    };
}

```

1.8 Virtual Threads (Java 21 - Project Loom)

Les **virtual threads** sont des threads légers gérés par la JVM, permettant d'exécuter des millions de threads simultanément sans explosion de ressources mémoire et CPU.

```

// Avant : threads lourds, pool limit
ExecutorService executor = Executors.newFixedThreadPool(100);
for (int i = 0; i < 1_000_000; i++) {
    executor.submit(() -> {
        // travail
        Thread.sleep(1000); // Bloque le thread physique
    });
}

// Après : virtual threads
ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor();
for (int i = 0; i < 1_000_000; i++) {
    executor.submit(() -> {
        // travail
        Thread.sleep(1000); // Ne bloque que le virtual thread
    });
}

// Plus simple avec fibres
Thread.ofVirtual()
    .name("worker-" + i)
    .start(() -> {

```

```
System.out.println("Virtual thread ID: " + Thread.currentThread()  
().threadId());  
});
```

1.9 Concepts Transversaux Importants

1.9.1 Immutabilité et Thread Safety

L'immutabilité est clé pour la programmation parallèle sûre. Un objet immuable ne peut pas être modifié après sa création :

- Tous les champs `final`
- Pas de setters
- Pas de retour de références mutables
- Records garantissent l'immutabilité

1.9.2 Composition vs Héritage

Préférer la composition (délégation) à l'héritage pour plus de flexibilité et moins de couplage.

```
// Mauvais : héritage rigide  
public class Voiture extends Vehicule {  
    // ...  
}  
  
// Bon : composition  
public class Voiture {  
    private Moteur moteur;  
    private Reservoir reservoir;  
  
    public void demarrer() {  
        moteur.demarrer();  
    }  
}
```

1.9.3 Principes SOLID

- **S - Single Responsibility** : une classe, une responsabilité
- **O - Open/Closed** : ouvert à l'extension, fermé à la modification
- **L - Liskov Substitution** : les sous-types doivent être substituables
- **I - Interface Segregation** : préférer des interfaces spécifiques à génériques
- **D - Dependency Inversion** : dépendre d'abstractions, pas de concrétions

2 Spring Boot : Architecture Complète et Bonnes Pratiques

2.1 Architecture en Couches Classique

L'architecture en couches est le standard industry pour structurer les applications web. Chaque couche possède une responsabilité bien définie :

- **Controller (Présentation)** : expose les endpoints REST, valide les inputs, appelle les services
- **Service (Logique Métier)** : orchestre les opérations, applique les règles métier, coordonne avec les repositories

- **Repository (Persistance)** : accès abstrait aux données, CRUD, requêtes personnalisées
- **Entity (Modèle de Données)** : représentation des données en base de données
- **DTO (Data Transfer Object)** : transfert de données entre couches, découplage API/interne

2.1.1 Pattern Service

Le service encapsule la logique métier complexe et facilite les tests unitaires :

```
@Service
@Transactional
public class UtilisateurService {

    private final UtilisateurRepository repository;
    private final EmailService emailService;

    @Autowired
    public UtilisateurService(UtilisateurRepository repo, EmailService
        email) {
        this.repository = repo;
        this.emailService = email;
    }

    public UtilisateurDTO creerUtilisateur(CreateUtilisateurRequest
        request) {
        // Validation m tier
        if (repository.existsByEmail(request.getEmail())) {
            throw new EmailAlreadyExistsException();
        }

        // Cr ation entit
        Utilisateur user = new Utilisateur();
        user.setEmail(request.getEmail());
        user.setMotDePasse(encodePassword(request.getMotDePasse()));

        // Persistance
        Utilisateur saved = repository.save(user);

        // Effets secondaires
        emailService.envoyerConfirmation(saved.getEmail());

        // DTO pour retour
        return mapToDTO(saved);
    }

    public Optional<UtilisateurDTO> obtenirParId(Long id) {
        return repository.findById(id).map(this::mapToDTO);
    }
}
```

2.1.2 DTOs et Conversion

Les DTOs (Data Transfer Objects) découplent l'API interne des entités JPA. Cela offre flexibilité lors de l'évolution du schéma ou des besoins des clients :

```
// Entit JPA
@Entity
```

```

@Table(name = "utilisateurs")
public class Utilisateur {
    @Id
    @GeneratedValue
    private Long id;

    private String email;
    private String motDePasse; // Jamais retourner !

    @OneToMany(mappedBy = "utilisateur")
    private Set<Commande> commandes;
}

// DTO pour la r p nse API
public record UtilisateurDTO(
    Long id,
    String email,
    int nombreCommandes
) {}

// Mapper
@Component
public class UtilisateurMapper {
    public UtilisateurDTO toDTO(Utilisateur user) {
        return new UtilisateurDTO(
            user.getId(),
            user.getEmail(),
            user.getCommandes().size()
        );
    }
}

```

2.2 Configuration Spring Boot

2.2.1 application.yml et Profils

Spring Boot utilise des fichiers de configuration pour gérer les environnements (dev, test, prod) sans recompiler le code :

```

# application.yml (commun)
spring:
  application:
    name: mon-api
  jpa:
    hibernate:
      ddl-auto: validate
    show-sql: false

# application-dev.yml
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/mydb_dev
    username: dev_user
    password: dev_password
  jpa:
    show-sql: true

# application-prod.yml

```

```

spring:
  datasource:
    url: jdbc:mysql://prod-db-server:3306/mydb
    username: ${DB_USER}
    password: ${DB_PASSWORD}
  jpa:
    show-sql: false

```

Lancer avec un profil spécifique :

```

# Command line
java -jar app.jar --spring.profiles.active=prod

# Properties
spring.profiles.active=dev

```

2.2.2 ConfigurationProperties

Pour des configurations typées et validées :

```

@Configuration
@ConfigurationProperties(prefix = "app.email")
public class EmailConfig {
    private String host;
    private int port;
    private String from;
    private boolean enableTls;

    // getters/setters
}

// application.yml
app:
  email:
    host: smtp.gmail.com
    port: 587
    from: noreply@company.com
    enableTls: true

// Injection
@Component
public class EmailService {
    @Autowired
    private EmailConfig config;
}

```

2.3 Persistance avec Spring Data JPA

2.3.1 Repository Pattern

Spring Data JPA fournit un framework pour l'accès aux données avec zéro ou minimal code boilerplate :

```

@Repository
public interface UtilisateurRepository extends JpaRepository<Utilisateur, Long> {
    // Requetes g n r es automatiquement par le nommage de m thode
    Optional<Utilisateur> findByEmail(String email);
}

```

```

List<Utilisateur> findByAgeGreaterThan(int age);
List<Utilisateur> findByVilleAndAgeOrderByNomAsc(String ville, int
    age);

// Requête personnalisée
@Query("SELECT u FROM Utilisateur u WHERE u.age > :age")
List<Utilisateur> findAdults(@Param("age") int age);

// Suppression personnalisée
@Modifying
@Query("DELETE FROM Utilisateur u WHERE u.email LIKE %:domain%")
int deleteByEmailDomain(@Param("domain") String domain);
}

```

2.3.2 Relations et Lazy Loading

```

// OneToMany : une personne, plusieurs commandes
@Entity
public class Utilisateur {
    @Id
    @GeneratedValue
    private Long id;

    @OneToMany(mappedBy = "utilisateur", fetch = FetchType.LAZY)
    private Set<Commande> commandes;
}

@Entity
public class Commande {
    @Id
    @GeneratedValue
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "utilisateur_id")
    private Utilisateur utilisateur;
}

// Attention au N+1 problem
// Au lieu de :
List<Utilisateur> users = repository.findAll(); // 1 query
for (Utilisateur u : users) {
    Set<Commande> cmds = u.getCommandes(); // N queries !
}

// Utiliser eager loading ou @Query avec join
@Query("SELECT DISTINCT u FROM Utilisateur u LEFT JOIN FETCH u.commandes
")
List<Utilisateur> findAllWithCommandes();

```

2.4 Gestion des Transactions

```

@Service
@Transactional // D marre transaction pour chaque méthode
public class TransferService {

```

```

@Transactional(readOnly = true)
public BigDecimal getSolde(Long compteId) {
    // Pas de modification, optimis
    return compteRepository.findById(compteId).getSolde();
}

@Transactional(propagation = Propagation.REQUIRED)
public void transferer(Long de, Long vers, BigDecimal montant) {
    Compte source = compteRepository.findById(de).orElseThrow();
    Compte dest = compteRepository.findById(vers).orElseThrow();

    source.debiter(montant);
    dest.crediter(montant);

    compteRepository.save(source);
    compteRepository.save(dest);
    // Si exception levée : ROLLBACK automatique
}
}

```

Niveaux d'isolation des transactions :

- **READ_UNCOMMITTED** : lectures sales (non recommandé)
- **READ_COMMITTED** : défaut, évite les lectures sales mais pas les fantômes
- **REPEATABLE_READ** : sérialisé mais lectures fantômes possibles
- **SERIALIZABLE** : isolation complète, performances réduites

2.5 Internationalisation (i18n)

```

// Configuration
@Configuration
public class I18nConfig {
    @Bean
    public MessageSource messageSource() {
        ResourceBundleMessageSource source = new
            ResourceBundleMessageSource();
        source.setBasename("messages");
        source.setDefaultEncoding("UTF-8");
        return source;
    }
}

// Fichiers properties
// messages_fr.properties
validation.email.required=L'email est obligatoire
validation.email.invalid=Email invalide

// messages_en.properties
validation.email.required=Email is required
validation.email.invalid=Invalid email

// Utilisation
@Component
public class ValideurUtilisateur {
    @Autowired
    private MessageSource messageSource;
}

```



```

    public String validerEmail(String email) {
        if (email == null) {
            return messageSource.getMessage(
                "validation.email.required",
                null,
                Locale.FRENCH
            );
        }
        return null;
    }
}

```

2.6 Gestion des Erreurs

```

// Exception personnalisée
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}

// Exception handler global
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(
        ResourceNotFoundException ex) {
        ErrorResponse error = new ErrorResponse(
            LocalDateTime.now(),
            404,
            "NOT_FOUND",
            ex.getMessage()
        );
        return ResponseEntity.status(404).body(error);
    }

    @ExceptionHandler(DataIntegrityViolationException.class)
    public ResponseEntity<ErrorResponse> handleDuplicate(
        DataIntegrityViolationException ex) {
        ErrorResponse error = new ErrorResponse(
            LocalDateTime.now(),
            409,
            "CONFLICT",
            "La ressource existe déjà"
        );
        return ResponseEntity.status(409).body(error);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneric(Exception ex) {
        ErrorResponse error = new ErrorResponse(
            LocalDateTime.now(),
            500,
            "INTERNAL_ERROR",
            "Erreur serveur interne"
        );
    }
}

```

```

        return ResponseEntity.status(500).body(error);
    }
}

```

2.7 Base de Données : Indexes et Schémas

2.7.1 Indexes pour Performances

Les indexes accélèrent les recherches mais ralentissent les écritures (insertion/mise à jour de l'index) :

```

@Entity
public class Utilisateur {
    @Id
    @GeneratedValue
    private Long id;

    @Column(unique = true)
    @Index(name = "idx_email")
    private String email;

    @Column
    @Index(name = "idx_age")
    private int age;

    // Index composite
    @Table(indexes = {
        @Index(name = "idx_ville_age", columnList = "ville,age")
    })
}

```

2.7.2 Versioning des Schémas

Utiliser Flyway ou Liquibase pour gérer l'évolution des schémas en production :

```

// Flyway : fichiers SQL classés par version
// db/migration/
// V1__initial_schema.sql
// V2__add_users_table.sql
// V3__add_email_index.sql

-- V3__add_email_index.sql
CREATE INDEX idx_email ON utilisateurs(email);
ALTER TABLE utilisateurs ADD COLUMN telephone VARCHAR(20);

```

3 Spring WebFlux : Programmation Réactive

3.1 Reactive Streams et Backpressure

Le modèle réactif s'oppose au modèle traditionnel bloquant. Caractéristiques :

- **Non-bloquant** : pas d'attente passive
- **Asynchrone** : résultats livrés quand prêts
- **Event-driven** : réagit aux événements
- **Backpressure** : le consumer contrôle le rythme de production

3.2 Mono et Flux : Types de Streams

- **Mono<T>** : 0 ou 1 élément
- **Flux<T>** : 0 à N éléments

```
// Mono
Mono<String> mono = Mono.just("hello");
Mono<String> empty = Mono.empty();
Mono<String> error = Mono.error(new RuntimeException());

// Flux
Flux<Integer> flux = Flux.just(1, 2, 3);
Flux<String> words = Flux.fromIterable(Arrays.asList("a", "b", "c"));
Flux<Long> interval = Flux.interval(Duration.ofSeconds(1));

// Souscription (n cessaire pour ex cution)
mono.subscribe(
    value -> System.out.println("Valeur: " + value),
    error -> System.err.println("Erreur: " + error),
    () -> System.out.println("Compl t ")
);
```

3.3 Opérateurs Réactifs Clés

```
Flux<Integer> flux = Flux.range(1, 10);

// map : transformation
flux.map(n -> n * 2) // 2, 4, 6, ...

// filter : filtrage
flux.filter(n -> n % 2 == 0) // 2, 4, 6, 8, 10

// flatMap : map asynchrone
flux.flatMap(n -> asyncOperationReturnsFlux(n))

// reduce : agr gation
flux.reduce((a, b) -> a + b) // Mono<Integer>

// collect : accumulation
flux.collect(Collectors.toList()) // Mono<List<Integer>>

// doOnNext : effets secondaires (logging, etc.)
flux.doOnNext(n -> System.out.println("Processing: " + n))

// onErrorResume : gestion erreur avec fallback
```

```

flux.onErrorResume(e -> Flux.just(0, 0, 0))

// timeout : limite de temps
flux.timeout(Duration.ofSeconds(5))

// retry : r essai automatique
flux.retry(3)

// subscribeOn : thread pour subscription
flux.subscribeOn(Schedulers.boundedElastic())

// publishOn : thread pour ex cution
flux.publishOn(Schedulers.parallel())

// cache : m morisation des r sultats
Flux<Integer> cached = flux.cache(Duration.ofSeconds(10));

// share : partage entre subscribers
Flux<Integer> shared = flux.share();

```

3.4 Contrôle de Concurrency

```

// limitRate : backpressure
Flux<Integer> limited = Flux.range(1, 1000)
    .limitRate(10); // Demande 10 la fois au lieu de tous

// concatWith : s quentiel
Mono<Integer> m1 = Mono.just(1);
Mono<Integer> m2 = Mono.just(2);
m1.concatWith(m2); // 1 puis 2

// mergeWith : parall le
m1.mergeWith(m2); // 1 et 2 en concurrence

// zipWith : combinaison
Mono<String> name = Mono.just("Alice");
Mono<Integer> age = Mono.just(30);
name.zipWith(age, (n, a) -> n + " (" + a + " ans)")
    // Alice (30 ans)

// combineLatest : dernier de chaque
Flux<Integer> nums = Flux.interval(Duration.ofMillis(100)).take(5);
Flux<String> strs = Flux.interval(Duration.ofMillis(150)).take(5).map(i
    -> "s" + i);
Flux.combineLatest(nums, strs, (n, s) -> n + s);

```

3.5 WebFlux : Contrôleurs Réactifs

```

@RestController
@RequestMapping("/api")
public class ReactiveController {

    @Autowired
    private ReactiveService service;

    // Mono retour

```

```

@GetMapping("/user/{id}")
public Mono<ResponseEntity<UserDTO>> getUser(@PathVariable Long id)
{
    return service.getUserById(id)
        .map(user -> ResponseEntity.ok(user))
        .onErrorResume(e -> Mono.just(ResponseEntity.notFound()).
            build());
}

// Flux retour (streaming)
@GetMapping("/users", produces = MediaType.APPLICATION_NDJSON_VALUE)
public Flux<UserDTO> getAllUsers() {
    return service.getAllUsers();
}

// Post r actif
@PostMapping("/users")
public Mono<ResponseEntity<UserDTO>> createUser(@RequestBody Mono<
    CreateUserRequest> requestMono) {
    return requestMono
        .flatMap(req -> service.createUser(req))
        .map(user -> ResponseEntity.status(201).body(user))
        .onErrorResume(e -> Mono.just(ResponseEntity.badRequest()).
            build());
}
}

\subsection{Service R actif}

\begin{lstlisting}
@Service
public class ReactivService {

    @Autowired
    private ReactiveUserRepository repo;

    public Mono<UserDTO> getUserById(Long id) {
        return repo.findById(id)
            .map(this::toDTO)
            .switchIfEmpty(Mono.error(new ResourceNotFoundException("
                User not found")));
    }

    public Flux<UserDTO> getAllUsers() {
        return repo.findAll()
            .map(this::toDTO)
            .timeout(Duration.ofSeconds(10));
    }

    public Mono<UserDTO> createUser(CreateUserRequest req) {
        return Mono.just(req)
            .map(r -> new User(r.getEmail(), r.getName()))
            .flatMap(repo::save)
            .map(this::toDTO);
    }

    // Combining multiple reactives
    public Mono<UserProfileDTO> getUserProfile(Long userId) {

```

```
        Mono<UserDTO> user = getUserById(userId);
        Mono<List<OrderDTO>> orders = getOrdersByUserId(userId).
            collectList();

        return Mono.zip(user, orders, (u, o) ->
            new UserProfileDTO(u, o)
        );
    }
}
```

4 Messaging et Événements : Kafka

4.1 Architecture et Concepts Fondamentaux

Kafka est une plateforme de streaming distribuée conçue pour publier, souscrire et traiter des flux d'événements à haute latence et grande échelle.

- **Producer (Producteur)** : publie les messages dans les topics
- **Consumer (Consommateur)** : consomme les messages d'un topic
- **Broker (Serveur)** : stocke et distribue les messages
- **Topic** : canal logique de messages, divisé en partitions
- **Partition** : unité de stockage parallélisable, ordonnée
- **Offset** : position séquentielle dans une partition
- **Consumer Group** : groupe de consommateurs lisant le même topic

4.2 Partitions et Parallélisme

Chaque topic contient plusieurs partitions. Les messages d'une même clé vont toujours à la même partition (ordre garantie), mais les partitions différentes sont indépendantes (parallélisme).

```
// Configuration d'un topic Kafka
Topic: commandes (3 partitions)
    Partition 0: commandes de utilisateur_1, utilisateur_4, ...
    Partition 1: commandes de utilisateur_2, utilisateur_5, ...
    Partition 2: commandes de utilisateur_3, utilisateur_6, ...

// Key d termine la partition : hash(key) % partitions
// Pas de key : distribution round-robin

Producer :
ProducerRecord record = new ProducerRecord<>(
    "commandes",
    "utilisateur_123",    // key : d termine partition
    commandeJSON         // value
);
```

4.3 Consumer Groups et Scalabilité

Un consumer group contient plusieurs consommateurs lisant le même topic. Kafka assure qu'un message n'est consommé que par un seul consumer du groupe.

```
// Consumer Group : ordre_processing_group
// 3 consommateurs, 3 partitions (optimal)
Topic: commandes (3 partitions)
    Partition 0 -> Consumer 1
    Partition 1 -> Consumer 2
    Partition 2 -> Consumer 3

// Scalabilit horizontale : ajouter consumer fait r assignation
// Sc nario : 2 partitions, 3 consumers
    Partition 0 -> Consumer 1
    Partition 1 -> Consumer 2
    Consumer 3 : inactif (rebalance possible      ajout partition)
```

4.4 Offsets et Commits

L'offset représente la position dans une partition. Kafka track les offsets consommés per consumer group.

```
// Offset tracking
Partition 0:
[0][1][2][3][4][5][6]... messages
    ^
    Consumer offset = 3 (prochaine lecture : 4)

// Auto commit (default)
@KafkaListener(topics = "commandes", groupId = "order_group")
public void consume(String message) {
    processCommande(message);
    // Offset autocommitt toutes les 5 secondes (default)
}

// Manual commit (plus sûr)
@KafkaListener(topics = "commandes", groupId = "order_group")
public void consumeManual(ConsumerRecord<String, String> record,
                          Acknowledgment ack) {
    try {
        processCommande(record.getValue());
        ack.acknowledge(); // Commit seulement si traitement réussi
    } catch (Exception e) {
        // Ne pas committer, retry prochaine lecture
        throw new ConsumerSeekToCurrentErrorHandler().handle(e, record);
    }
}
```

4.5 Intégration Spring Kafka

```
// Configuration
@Configuration
public class KafkaConfig {
    @Bean
    public KafkaTemplate<String, String> kafkaTemplate(
        ProducerFactory<String, String> producerFactory) {
        return new KafkaTemplate<>(producerFactory);
    }
}

// Producer
@Service
public class CommandeProducer {
    @Autowired
    private KafkaTemplate<String, String> kafkaTemplate;

    public void envoyerCommande(Commande cmd) {
        String json = convertToJson(cmd);
        kafkaTemplate.send("commandes", cmd.getUtilisateurId().toString(), json)
            .addCallback(
                result -> System.out.println("Message envoyé : " + result.getRecordMetadata()),
                ex -> System.err.println("Erreur: " + ex)
            );
    }
}
```



```

    }
}

// Consumer
@Component
public class CommandeConsumer {
    @KafkaListener(
        topics = "commandes",
        groupId = "order_processing",
        containerFactory = "kafkaListenerContainerFactory"
    )
    public void consommer(ConsumerRecord<String, String> record) {
        Commande cmd = convertFromJson(record.getValue());
        traitementCommande(cmd);
    }
}

```

4.6 Garanties et Sémantiques de Livraison

- **At Most Once** : risque de perte de messages
- **At Least Once** : risque de duplication, pas de perte
- **Exactly Once** : garantie complète (plus complexe, overhead)

Configuration pour At Least Once :

```

# application.yml
spring:
  kafka:
    producer:
      acks: all          # Attend confirmation tous replicas
      retries: 3
      max-in-flight-requests-per-connection: 1 # Ordre garanti
    consumer:
      auto-offset-reset: earliest # Relire depuis d but si pas offset
      enable-auto-commit: false   # Manual commit

```

5 Cloud et Infrastructure : AWS Essentials

5.1 Compute Services

5.1.1 EC2 : Machines Virtuelles Élastiques

EC2 (Elastic Compute Cloud) fournit des serveurs virtuels à la demande :

- **Instance Types** : t2.micro, t3.small, m5.large, c5.2xlarge, etc.
- **t = Burstable** : performances variables, bon pour trafic imprévisible
- **m = General Purpose** : équilibre CPU/mémoire
- **c = Compute Optimized** : CPU-intensif
- **r = Memory Optimized** : bases données, caches

5.1.2 Containerization : Docker et ECS

Docker encapsule une application avec ses dépendances dans un conteneur portable et immuable :

```
# Dockerfile
FROM openjdk:21-slim

WORKDIR /app

COPY target/myapp.jar application.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "application.jar"]

# Build
docker build -t myapp:1.0 .

# Run
docker run -p 8080:8080 myapp:1.0
```

ECS (Elastic Container Service) orchestre les conteneurs Docker sur des clusters EC2 ou Fargate (serverless containers) :

- Déploiement et scaling automatiques
- Service discovery
- Load balancing
- Logging centralisé

5.1.3 ECR : Registry Docker Managé

ECR (Elastic Container Registry) stocke et distribue les images Docker de votre organisation.

5.2 Storage Services

5.2.1 S3 : Stockage Objet Illimité

S3 (Simple Storage Service) fournit un stockage objet hautement disponible et durable :

- Coût faible par GB
- Durabilité 99.999999999% (11 nines)

- Scaling illimité
- Accès par clé (path), pas par filesystem
- Politiques d'accès granulaires (IAM, bucket policies)

Cas d'usage : fichiers utilisateur, backups, contenu statique, logs d'application.

5.3 Messaging : SQS et SNS

5.3.1 SQS : File de Messages Découplée

SQS (Simple Queue Service) découple les producteurs et consommateurs de messages :

- FIFO vs Standard queues
- Délai de livraison configurable
- Visibility timeout (délai avant relivraison si pas ack)
- Long polling pour efficacité

5.3.2 SNS : Publication/Souscription

SNS (Simple Notification Service) permet la pub/sub scalable d'événements :

- Fan-out à plusieurs souscripteurs
- Nombreux protocoles : email, SMS, SQS, HTTP, Lambda
- Combinable avec SQS (fan-out pattern)

5.4 CloudFront : CDN Global

CloudFront distribue votre contenu via un réseau mondial de data centers (edge locations) :

- Réduction de latence pour utilisateurs mondiaux
- Décharges votre origine (cache)
- DDoS protection intégré
- SSL/TLS par défaut
- Intégration S3, ALB, custom origins

6 CI/CD avec GitLab

6.1 Pipelines et Stages

Une **pipeline** GitLab orchestre des jobs à travers plusieurs stages. Chaque stage ne s'exécute que si le stage précédent réussit.

```
# .gitlab-ci.yml

stages:
  - build
  - test
  - deploy-staging
  - deploy-prod

build:
  stage: build
  script:
    - echo "Building..."
    - mvn clean package
  artifacts:
    paths:
      - target/myapp.jar

test:unit:
  stage: test
  script:
    - mvn test
  coverage: '/Coverage: \d+\.\d+%/'

deploy_staging:
  stage: deploy-staging
  script:
    - echo "Deploying to staging..."
    - aws s3 cp target/myapp.jar s3://staging-bucket/
  only:
    - develop

deploy_prod:
  stage: deploy-prod
  script:
    - echo "Deploying to production..."
    - aws s3 cp target/myapp.jar s3://prod-bucket/
  only:
    - main
  when: manual # Approbation manuelle requise
```

6.2 Runners : Exécution Distribuée

Un **runner** est un serveur qui exécute les jobs définis dans la pipeline.

- **Shared Runners** : fournis par GitLab, partagés entre tous les projets
- **Specific Runners** : déployés privément pour des besoins spécifiques
- **Docker Runner** : exécute jobs dans des conteneurs Docker
- **Kubernetes Runner** : exécute dans un cluster K8s

6.3 Advanced Patterns

```
# Environnements et variables
deploy_prod:
  stage: deploy-prod
  environment:
    name: production
    url: https://api.example.com
  variables:
    DB_HOST: prod-db.example.com
    DEPLOY_ENV: production
  script:
    - echo "Deploying to $DEPLOY_ENV"
    - export APP_VERSION=$CI_COMMIT_SHA
    - docker build -t myapp:$APP_VERSION .

# D pendances entre jobs
generate_report:
  stage: test
  script:
    - create_report.sh
  artifacts:
    paths:
      - report.pdf
    expire_in: 1 week

publish_report:
  stage: deploy-staging
  dependencies:
    - generate_report
  script:
    - upload_report.sh report.pdf

# Cache pour performances
build:
  stage: build
  cache:
    paths:
      - .m2/
      - target/
  script:
    - mvn clean package
```

7 Frontend : Angular Moderne

7.1 Composants et Data Binding

```
// Component
@Component({
  selector: 'app-user-list',
  template: `
    <div *ngFor="let user of users$ | async">
      {{ user.name }} - {{ user.email }}
    </div>
  `,
  styles: [`
    div { padding: 10px; border: 1px solid #ccc; }
  `]
})
export class UserListComponent implements OnInit {
  users$: Observable<User []>;

  constructor(private userService: UserService) {}

  ngOnInit() {
    this.users$ = this.userService.getAll();
  }
}
```

7.2 RxJS et Observables

RxJS est la librairie de programmation réactive pour Angular :

```
// Service avec Observable
@Injectable()
export class UserService {
  constructor(private http: HttpClient) {}

  getAll(): Observable<User []> {
    return this.http.get<User []>('/api/users').pipe(
      tap(users => console.log('Users loaded:', users)),
      catchError(error => {
        console.error('Error loading users', error);
        return throwError(() => new Error('Cannot load users'));
      })
    );
  }

  getById(id: number): Observable<User> {
    return this.http.get<User>('/api/users/${id}');
  }
}

// Component souscription
export class UserDetailComponent implements OnInit, OnDestroy {
  user$: Observable<User>;
  private destroy$ = new Subject<void>();

  constructor(private userService: UserService, private route:
    ActivatedRoute) {}
}
```

```

ngOnInit() {
  this.user$ = this.route.params.pipe(
    switchMap(params => this.userService.getById(params['id'])),
    takeUntil(this.destroy$)
  );
}

ngOnDestroy() {
  this.destroy$.next();
  this.destroy$.complete();
}
}

```

Opérateurs RxJS essentiels :

- `map` : transformation
- `switchMap` : annule observable précédent, lance nouveau
- `mergeMap` : lance parallèle
- `concatMap` : en séquence
- `filter` : filtrage
- `tap` : effet secondaire sans modification
- `catchError` : gestion erreur
- `takeUntil` : arrête si condition remplie
- `debounceTime` : attend avant d'émettre
- `distinctUntilChanged` : ignore doublons consécutifs

7.3 Pagination

7.3.1 Backend Pagination

```

// API response
@GetMapping("/users")
public Page<UserDTO> listUsers(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "10") int size,
    @RequestParam(defaultValue = "id,desc") String[] sort
) {
    Sort sortBy = Sort.by(Sort.Direction.DESC, "id");
    PageRequest pageable = PageRequest.of(page, size, sortBy);
    return userService.getAllPaginated(pageable)
        .map(this::toDTO);
}

// Response JSON
{
  "content": [...],
  "pageable": {
    "pageNumber": 0,
    "pageSize": 10
  },
  "totalElements": 250,
  "totalPages": 25,
  "last": false,
  "first": true,
  "number": 0,

```

```

    "size": 10,
    "numberOfElements": 10
  }
}

```

7.3.2 Frontend Pagination

```

// Component avec pagination
export class UserListComponent implements OnInit {
  users$: Observable<User[]>;
  total$: Observable<number>;

  pageIndex = 0;
  pageSize = 10;

  constructor(private userService: UserService) {}

  ngOnInit() {
    this.loadUsers();
  }

  loadUsers() {
    this.users$ = this.userService.getUsers(this.pageIndex, this.pageSize)
      .pipe(
        tap(page => this.total$ = of(page.totalElements)),
        map(page => page.content)
      );
  }

  onPageChange(event: PageEvent) {
    this.pageIndex = event.pageIndex;
    this.pageSize = event.pageSize;
    this.loadUsers();
  }
}

// Template
<table>
  <tr *ngFor="let user of users$ | async">
    <td>{{ user.name }}</td>
  </tr>
</table>

<mat-paginator
  [length]="total$ | async"
  [pageSize]="pageSize"
  [pageSizeOptions]="[5, 10, 25]"
  (page)="onPageChange($event)">
</mat-paginator>

```

7.4 Guards et Authentification

```

// Auth Guard
@Injectable()
export class AuthGuard implements CanActivate {
  constructor(

```



```

    private authService: AuthService,
    private router: Router
  ) {}

  canActivate(
    route: ActivatedRouteSnapshot,
    state: RouterStateSnapshot
  ): Observable<boolean> {
    return this.authService.isAuthenticated().pipe(
      switchMap(isAuth => {
        if (isAuth) {
          const requiredRole = route.data['role'];
          return this.authService.hasRole(requiredRole);
        }
        return of(false);
      }),
      tap(allowed => {
        if (!allowed) {
          this.router.navigate(['/login']);
        }
      })
    );
  }
}

// Route configuration
const routes: Routes = [
  {
    path: 'admin',
    component: AdminDashboardComponent,
    canActivate: [AuthGuard],
    data: { role: 'ADMIN' }
  },
  {
    path: 'user',
    component: UserDashboardComponent,
    canActivate: [AuthGuard],
    data: { role: 'USER' }
  }
];

```

7.5 Signals et Effects (Angular 17+)

Les Signals offrent une alternative à RxJS pour la réactivité simple :

```

// Signal : tat r actif
export class CounterComponent {
  count = signal(0);

  increment() {
    this.count.update(c => c + 1);
  }

  // Computed signal
  doubled = computed(() => this.count() * 2);
}

// Template

```

```

<div>{{ count() }} (doub : {{ doubled() }})</div>

// Effect : r action automatique
export class UserComponent implements OnInit {
  userId = signal<number | null>(null);
  user = signal<User | null>(null);

  constructor(private userService: UserService) {
    effect(() => {
      const id = this.userId();
      if (id) {
        this.userService.getById(id).subscribe(
          user => this.user.set(user)
        );
      }
    });
  }
}

```

7.6 Change Detection et Performance Angular

7.6.1 Change Detection Strategy

Angular par défaut vérifie éventuellement tout changement lors d'événements. Pour optimiser :

```

// D faut : v rifie chaque vnement
@Component({
  selector: 'app-expensive',
  template: '{{ compute() }}'
})
export class ExpensiveComponent {}

// Optimis : v rifie seulement si @Input change
@Component({
  selector: 'app-optimized',
  template: '{{ data.name }}',
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class OptimizedComponent {
  @Input() data: User;

  constructor(private cdr: ChangeDetectorRef) {}

  // Force v rification si n cessaire
  updateData() {
    this.cdr.markForCheck();
  }
}

```

7.6.2 TrackBy avec *ngFor

Sans trackBy, Angular détruit et recrée chaque élément lors de modification de la liste :

```

// Inefficace
<div *ngFor="let item of items">{{ item.name }}</div>

// Efficace : r utilise instances pour m mes ID

```

```

<div *ngFor="let item of items; trackBy: trackByFn">{{ item.name }}</div>
>

trackByFn(index: number, item: Item): number {
  return item.id; // Identifiant unique
}

```

7.7 Concurrency et Combinaison Observables

```

// forkJoin : attendez tout, fail si une chose
export class DashboardComponent implements OnInit {
  user$: Observable<User>;
  stats$: Observable<Stats>;
  orders$: Observable<Order[]>;

  dashboard$: Observable<{user: User, stats: Stats, orders: Order[]}>;

  ngOnInit() {
    this.dashboard$ = forkJoin({
      user: this.userService.getCurrentUser(),
      stats: this.analyticsService.getStats(),
      orders: this.orderService.getUserOrders()
    }).pipe(
      catchError(error => {
        this.showError('Impossible de charger le tableau de bord');
        return EMPTY;
      })
    );
  }
}

// combineLatest : met quand au moins une source change
export class FilteredListComponent {
  category$ = new BehaviorSubject('all');
  search$ = new Subject<string>();

  filteredProducts$ = combineLatest([
    this.category$,
    this.search$.pipe(debounceTime(300), distinctUntilChanged())
  ]).pipe(
    switchMap(([category, search]) =>
      this.productService.search(category, search)
    )
  );
}

// race : premi ère source mettre gagne
Promise.race([
  this.userService.getUser(),
  timeout(5000)
]).then(user => console.log('User loaded in time'));

```

8 Concurrency et Programmation Distribuée

8.1 Modèles de Concurrency

8.1.1 Thread Safety et Visibilité

Dans un environnement multi-thread, sans synchronisation, les modifications d'une thread ne sont pas visibles aux autres :

```
// Dangereux : data race
public class Counter {
    private int count = 0; // Sans volatile, lecture peut être cachée

    public void increment() { count++; }
    public int getCount() { return count; }
}

// Sécurisé
public class SafeCounter {
    private volatile int count = 0; // Chaque thread voit la dernière valeur

    public synchronized void increment() { count++; }
    public int getCount() { return count; }
}

// Mieux : Atomic
public class AtomicCounter {
    private final AtomicInteger count = new AtomicInteger(0);

    public void increment() { count.incrementAndGet(); } // Lock-free
    public int getCount() { return count.get(); }
}
```

8.1.2 Deadlock et Solutions

Deadlock se produit quand deux threads attendent mutuellement. Stratégies de prévention :

- **Ordering** : toujours acquérir les locks dans le même ordre
- **Timeout** : renoncer si lock non acquis dans le délai
- **Lock-free** : structures de données sans lock (ConcurrentHashMap)

8.1.3 Executor Framework

```
// ThreadPool réutilisable
ExecutorService executor = Executors.newFixedThreadPool(5);

for (int i = 0; i < 100; i++) {
    executor.submit(() -> {
        // travail
    });
}

executor.shutdown(); // N'accepte plus new tasks
executor.awaitTermination(10, TimeUnit.SECONDS);

// Future : résultat asynchrone
```

```
Future<Integer> future = executor.submit(() -> computeValue());
try {
    Integer result = future.get(5, TimeUnit.SECONDS); // Bloque avec
                                                    timeout
} catch (TimeoutException e) {
    future.cancel(true); // Annuler si d passe timeout
}
```

9 Patterns et Bonnes Pratiques Transversales

9.1 Gestion des Erreurs et Logging

9.1.1 Backend : Logging Structuré

```
// Configuration logback
<!-- logback.xml -->
<configuration>
    <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
        <layout class="ch.qos.logback.classic.PatternLayout">
            <pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n
            </pattern>
        </layout>
    </appender>

    <root level="INFO">
        <appender-ref ref="STDOUT"/>
    </root>
</configuration>

// Utilisation
@Service
public class OrderService {
    private static final Logger logger = LoggerFactory.getLogger(
        OrderService.class);

    public Order createOrder(CreateOrderRequest req) {
        logger.info("Creating order for user {}", req.getUserId());

        try {
            // ... logique
            logger.debug("Order details: {}", order);
            return order;
        } catch (InvalidOrderException e) {
            logger.error("Failed to create order for user {}", req.getUserId(), e);
            throw new OrderProcessingException("Order creation failed");
        }
    }
}
```

9.1.2 Frontend : Gestion Centralisée

```
// Error handler global
@Injectable()
export class GlobalErrorHandler implements ErrorHandler {
```

```

constructor(
    private notificationService: NotificationService,
    private logger: LoggerService
) {}

handleError(error: Error | HttpErrorResponse) {
    let message = 'Une erreur est survenue';

    if (error instanceof HttpErrorResponse) {
        message = this.getHttpErrorMessage(error);
    } else {
        this.logger.logError(error);
    }

    this.notificationService.error(message);
}

private getHttpErrorMessage(error: HttpErrorResponse): string {
    switch (error.status) {
        case 401:
            return 'Non authentifi ';
        case 403:
            return 'Acc s refus ';
        case 404:
            return 'Ressource non trouv e';
        case 500:
            return 'Erreur serveur';
        default:
            return error.error?.message || 'Erreur r seau';
    }
}
}

```

9.2 Testing

9.2.1 Backend : Tests Unitaires et Intégration

```

// Test unitaire
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {

    @Mock
    private OrderRepository orderRepository;

    @InjectMocks
    private OrderService orderService;

    @Test
    void should_create_order_successfully() {
        // Arrange
        CreateOrderRequest request = new CreateOrderRequest(1L, "item1");
        Order expectedOrder = new Order();

        when(orderRepository.save(any(Order.class)))
            .thenReturn(expectedOrder);

        // Act
    }
}

```

```

    Order result = orderService.createOrder(request);

    // Assert
    assertNotNull(result);
    verify(orderRepository, times(1)).save(any(Order.class));
}
}

// Test d'integration
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.
    RANDOM_PORT)
class OrderControllerIntegrationTest {

    @Autowired
    private TestRestTemplate restTemplate;

    @Test
    void should_create_order_via_rest() {
        CreateOrderRequest request = new CreateOrderRequest(1L, "item1");

        ResponseEntity<OrderDTO> response = restTemplate.postForEntity(
            "/api/orders",
            request,
            OrderDTO.class
        );

        assertEquals(HttpStatus.CREATED, response.getStatusCode());
        assertNotNull(response.getBody());
    }
}

```

9.2.2 Frontend : Tests Angular

```

// Jasmine + Karma
describe('UserListComponent', () => {
    let component: UserListComponent;
    let fixture: ComponentFixture<UserListComponent>;
    let userService: UserService;

    beforeEach(async () => {
        await TestBed.configureTestingModule({
            declarations: [ UserListComponent ],
            providers: [ { provide: UserService, useValue: { getAll: () => of
                ([]) } } ]
        }).compileComponents();

        fixture = TestBed.createComponent(UserListComponent);
        component = fixture.componentInstance;
        userService = TestBed.inject(UserService);
    });

    it('should display users list', (done) => {
        const users = [{ id: 1, name: 'Alice' }];
        spyOn(userService, 'getAll').and.returnValue(of(users));

        component.ngOnInit();
    });
}

```

```

        fixture.detectChanges();
        fixture.whenStable().then(() => {
            const rows = fixture.debugElement.queryAll(By.css('table tr'));
            expect(rows.length).toBe(1);
            done();
        });
    });
});

```

9.3 Stratégies de Cache et Invalidation

9.3.1 Backend Caching

```

// Cache local avec Caffeine
@Configuration
public class CacheConfig {
    @Bean
    public CacheManager cacheManager() {
        CaffeineCacheManager cacheManager = new CaffeineCacheManager();
        cacheManager.setCaffeine(Caffeine.newBuilder()
            .expireAfterWrite(10, TimeUnit.MINUTES)
            .recordStats());
        return cacheManager;
    }
}

// Utilisation
@Service
public class UserService {
    @Cacheable("users")
    public User getUser(Long id) {
        return userRepository.findById(id).orElseThrow();
    }

    @CacheEvict("users")
    public void updateUser(User user) {
        userRepository.save(user);
    }
}

// Cache distribu avec Redis
@Bean
public RedisCacheManager cacheManager(RedisConnectionFactory
    connectionFactory) {
    return RedisCacheManager.create(connectionFactory);
}

```

9.3.2 Frontend Caching Strategy

```

// Cache avec HttpClient et interceptor
@Injectable()
export class CacheInterceptor implements HttpInterceptor {
    private cache = new Map<string, any>();

    intercept(req: HttpRequest<any>, next: HttpHandler): Observable<
        HttpEvent<any>> {

```



```

// Seulement GET en cache
if (req.method !== 'GET') {
    return next.handle(req);
}

if (this.cache.has(req.url)) {
    return of(new HttpResponse({ body: this.cache.get(req.url)
    }));
}

return next.handle(req).pipe(
    tap(event => {
        if (event instanceof HttpResponse) {
            this.cache.set(req.url, event.body);
        }
    })
);
}
}

```

9.4 Performance et Optimisation

9.4.1 Backend

- **Connection pooling** : réutiliser connexions DB, HikariCP par défaut
- **Query optimization** : indexes, eager/lazy loading, N+1 problem
- **Caching** : Redis, Caffeine, stratégies d'invalidation
- **Async processing** : @Async, reactive streams, threads dédiés
- **Circuit breaker** : Resilience4j pour appels externes
- **Pagination** : limiter résultats, éviter full table scans
- **Profiling** : JProfiler, Prometheus métriques

9.4.2 Frontend

- **Change detection OnPush** : moins de détections manuelles, gain 5-10x
- **TrackBy avec *ngFor** : réutiliser instances DOM
- **Lazy loading modules** : charger features à la demande via routing
- **Image optimization** : compression (WebP), lazy loading images
- **Bundle analysis** : webpack-bundle-analyzer, tree-shaking
- **Virtual scrolling** : CDK virtual-scroll pour grandes listes
- **Unsubscribe** : toujours se désabonner des Observables
- **Async pipe** : gère subscription/unsubscription automatiquement

9.5 Debugging et Monitoring

9.5.1 Backend Monitoring

```

// Prometheus m triques
@Configuration
public class MetricsConfig {
    @Bean
    public MeterBinder customMetrics() {

```

```

        return (registry) -> {
            Gauge.builder("orders.active", () -> orderService.
                countActive())
                .register(registry);
            Counter.builder("orders.created")
                .description("Nombre de commandes crées")
                .register(registry);
        };
    }
}

// Health checks
@Component
public class DatabaseHealthIndicator extends AbstractHealthIndicator {
    @Override
    protected void doHealthCheck(Health.Builder builder) throws
        Exception {
        try {
            dataSource.getConnection().isValid(2);
            builder.up();
        } catch (Exception e) {
            builder.down().withException(e);
        }
    }
}

```

9.5.2 Frontend Debugging

- **Chrome DevTools** : Network tab pour API calls, Performance tab pour recording
- **Redux DevTools** : time-travel debugging si state management
- **Angular DevTools** : browser extension pour inspection composants
- **Lighthouse** : audit performance, accessibility, SEO

9.6 Sécurité et Compliance AWS

9.6.1 IAM : Contrôle d'Accès Granulaire

IAM (Identity and Access Management) gère les utilisateurs, rôles et permissions AWS. Principes fondamentaux :

- **Least Privilege** : donner seulement les permissions minimales nécessaires
- **Roles** : ensembles de permissions réutilisables
- **Policies** : documents JSON définissant les permissions
- **MFA** : authentification multi-facteur pour protection

Exemple de policy restrictive :

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": "arn:aws:s3:::my-bucket/*"
    }
  ]
}
```

9.6.2 VPC et Networking

VPC (Virtual Private Cloud) isole vos ressources AWS :

- **Subnets** : zones logiques publiques ou privées
- **Security Groups** : pare-feu au niveau instance
- **Network ACLs** : pare-feu au niveau subnet
- **NAT Gateway** : sortie internet contrôlée pour instances privées

9.6.3 Encryption et Secrets

- **KMS** : gestion des clés de chiffrement
- **Secrets Manager** : stockage sécurisé des secrets d'application
- **SSL/TLS** : chiffrement en transit

9.7 Patterns AWS Avancés

9.7.1 Auto Scaling

La scalabilité automatique ajuste le nombre d'instances selon la demande :

```
// Auto Scaling Group configuration
Min capacity: 2
Desired capacity: 5
Max capacity: 20
Scale-up policy: CPUUtilization > 70\%
Scale-down policy: CPUUtilization < 30\%
Cooldown period: 5 minutes
```

9.7.2 Load Balancing

Application Load Balancer (ALB) distribue le trafic :

- **Health checks** : vérifie instances saines
- **Path-based routing** : `/api/*` vers service A, `/images/*` vers service B
- **Sticky sessions** : route client vers même instance
- **SSL termination** : décrypte HTTPS au load balancer

9.8 Cost Optimization AWS

- **Reserved Instances** : 30-70% moins cher pour engagement 1-3 ans
- **Spot Instances** : 70-90% réduction pour workloads interruptibles
- **Right-sizing** : instance types adaptées à la charge réelle
- **Data transfer** : minimiser transferts inter-région
- **Storage optimization** : S3 Intelligent-Tiering, Glacier pour archives

10 Patterns d'Architecture Distribuée

10.1 CQRS et Event Sourcing

CQRS (Command Query Responsibility Segregation) sépare les opérations de lecture et écriture :

```
// Service avec CQRS
@Service
public class OrderService {

    // Write model : cr e des vnements
    @Transactional
    public void createOrder(CreateOrderCommand cmd) {
        Order order = new Order(cmd.getUserId(), cmd.getItems());
        repository.save(order);
        eventBus.publish(new OrderCreatedEvent(order.getId(), cmd.getItems()));
    }

    // Read model : requ te optimis e
    public List<OrderSummary> getUserOrders(Long userId) {
        // Requ te directe sur mat rialisation, pas de joins complexes
        return orderSummaryRepository.findByUserId(userId);
    }
}
```

10.2 Saga Pattern pour Transactions Distribuées

Les Sagas orchestrent les transactions distribuées sans locks globaux :

```
// Exemple : processus de commande multi-service
public class OrderSaga {

    public void executeOrderFlow(Order order) {
        // tape 1 : cr er commande
        orderService.createOrder(order);
        eventBus.publish(new OrderCreatedEvent(order.getId()));
    }
}
```

```

// tape 2 : r server inventaire
try {
    inventoryService.reserve(order.getItems());
    eventBus.publish(new InventoryReservedEvent());
} catch (InsufficientStockException e) {
    // Compensation : annuler commande
    orderService.cancel(order.getId());
    eventBus.publish(new OrderCancelledEvent());
    throw e;
}

// tape 3 : traiter paiement
try {
    paymentService.charge(order.getTotal());
    eventBus.publish(new PaymentProcessedEvent());
} catch (PaymentFailedException e) {
    // Compensation : annuler et lib rer inventaire
    orderService.cancel(order.getId());
    inventoryService.release(order.getItems());
    throw e;
}
}
}

```

10.3 Resilience Pattern

10.3.1 Circuit Breaker

Le Circuit Breaker empêche les cascades de défaillance :

```

@Service
public class PaymentService {

    @CircuitBreaker(name = "paymentService", fallbackMethod = "fallback"
    )
    public void processPayment(Payment payment) {
        externalPaymentAPI.charge(payment);
    }

    public void fallback(Payment payment, Exception e) {
        // Fallback : queue pour retry asynchrone
        pendingPaymentsQueue.add(payment);
        logger.error("Payment queued for retry: " + payment.getId(), e);
    }
}

```

11 Questions-Réponses : Préparation aux Entretiens

Cette section simule les questions les plus fréquemment posées lors d'entretiens techniques, avec des réponses structurées et orientées vers la démonstration de compétences pratiques.

11.1 Questions Java et Programmation Fonctionnelle

11.1.1 Q1 : Quelle est la différence entre List et Stream en Java ?

Réponse : Une List est une collection concrète et mutée en mémoire, tandis qu'un Stream est une abstraction lazy et immuable pour traiter les données. Les Streams ne modifient pas la source originale et utilisent l'évaluation lazy (les opérations intermédiaires ne s'exécutent que lorsqu'une opération terminale est invoquée). Les Streams sont plus performants pour traiter de grandes données ou appliquer des transformations complexes, car ils optimisent automatiquement les opérations. Cependant, une List permet l'accès aléatoire et la modification, tandis qu'un Stream est à usage unique et détruit après consommation.

11.1.2 Q2 : Pourquoi utiliser Optional plutôt que simplement vérifier null ?

Réponse : Optional force explicitement le développeur à traiter le cas absence de valeur, éliminant les NullPointerExceptions inattendues. C'est une forme de programmation défensive qui rend le code plus lisible et maintenable. Optional exprime l'intention que la valeur peut être absente, tandis qu'une simple vérification null est facile à oublier. De plus, Optional offre des opérations chaînées (map, flatMap, filter) qui rendent le code plus fonctionnel et expressif.

11.1.3 Q3 : Expliquez la différence entre parallel() et Sequential() dans les Streams

Réponse : Un Stream séquentiel traite les éléments un par un sur un seul thread. Un parallel Stream divise les données et les traite sur plusieurs threads via un fork/join pool. Le parallel Stream est plus rapide pour les collections volumineuses (>1000 éléments) et opérations CPU-intensives. Cependant, il introduit de l'overhead et du coût de synchronisation, donc pour petites données il est plus lent. Important : éviter les opérations stateful ou les I/O bloquants avec parallel Streams, car cela provoque des race conditions et diminue les performances.

11.1.4 Q4 : Qu'est-ce qu'une interface fonctionnelle et pourquoi c'est important ?

Réponse : Une interface fonctionnelle contient exactement une méthode abstraite et peut être implémentée via lambda expressions (syntaxe concise). C'est important parce que cela rend le code plus readable et fonctionnel. Java fournit des interfaces fonctionnelles prédéfinies (Predicate, Function, Consumer, Supplier) réutilisables pour la plupart des cas. Les lambdas permettent aussi de passer des comportements en argument, favorisant la programmation déclarative au lieu d'impérative.

11.1.5 Q5 : Quels sont les avantages des Records par rapport aux classes ordinaires ?

Réponse : Les Records génèrent automatiquement le constructeur, les accesseurs, equals(), hashCode() et toString(), réduisant considérablement le boilerplate. Ils garantissent l'immuabilité (tous les champs sont final), idéal pour les DTOs et valeurs immuables. Les Records rendent le code plus concis et moins sujet aux bugs (moins de code = moins d'erreurs). Ils sont aussi mieux pour la sérialisation et optimisés par la JVM (inline par exemple). Limitation : pas héritage entre Records, destinés surtout aux données simples.

11.1.6 Q6 : Expliquez comment Virtual Threads résout le problème de scalabilité

Réponse : Les threads traditionnels sont lourds (consommant 1-2 MB mémoire chacun), limitant le nombre qu'on peut créer (quelques milliers maximum). Virtual Threads sont légers (quelques KB) et gérés par la JVM, permettant millions de threads avec peu de ressources. Quand un Virtual Thread effectue une opération bloquante (sleep, I/O), la JVM le suspend et passe à un autre, sans gaspiller un thread OS. Cela permet une scalabilité massive sans refactoring en programmation réactive (Reactive Streams, Mono/Flux).

11.1.7 Q7 : Quel est l'impact du choix de collection sur les performances ?

Réponse : ArrayList offre accès $O(1)$ mais insertion/suppression au milieu $O(n)$. LinkedList offre insertion/suppression $O(1)$ au début mais accès $O(n)$. HashSet/HashMap offrent $O(1)$ moyen mais pas d'ordre. TreeSet/TreeMap offrent $O(\log n)$ et maintiennent l'ordre. Le choix dépend du cas d'usage : lecture fréquente = ArrayList, modifications fréquentes au début = LinkedList, recherche rapide = HashSet, ordre nécessaire = TreeSet. Aussi important : utiliser la capacité initiale correcte pour ArrayList (new ArrayList(expectedSize)) pour éviter redimensionnements coûteux.

11.1.8 Q8 : Qu'est-ce qu'une race condition et comment l'éviter ?

Réponse : Une race condition se produit quand deux threads accèdent à la même ressource sans synchronisation et au moins un modifie. Résultat : comportement imprévisible. Exemples : deux threads incrémentent un compteur simultanément (count++), résultat peut être erroné. Solutions : synchronized pour mutual exclusion, volatile pour visibilité mémoire, Atomic (AtomicInteger) pour opérations lock-free, ou Collections.synchronizedList pour collections thread-safe. Mieux : utiliser des structures concurrent (ConcurrentHashMap) au lieu de HashMap synchronisé (plus performant).

11.1.9 Q9 : Décrivez le cycle de vie des objets en Java

Réponse : Création : allocation mémoire et appel constructeur. Vie : utilisation par application. Garbage Collection : quand aucune référence pointe vers l'objet, GC le récupère. Finalization (déprécié) : finalize() appelée avant destruction. Destruction : mémoire retournée au heap. Important : ne pas dépendre sur finalize, préférer try-with-resources pour nettoyage de ressources. GC est automatique mais peut être déclenché (System.gc()), pas garanti) ou tuné via flags JVM.

11.1.10 Q10 : Qu'est-ce que l'immuabilité et pourquoi c'est important en concurrence ?

Réponse : L'immuabilité signifie qu'un objet ne peut pas être modifié après création. Avantages : aucune synchronisation nécessaire (threads peuvent accéder sans locks), cache-friendly (valeurs constantes), idéal pour keys HashMap. Implémentation : tous champs final, pas de setters, retourner copies au lieu de références. Exemple : String est immutable en Java, donc thread-safe. Désavantage : créer copies pour chaque modification peut être coûteux mémoire, mais généralement worth pour la sécurité concurrence et code plus simple.

11.2 Questions Spring Boot et Architecture Backend

11.2.1 Q1 : Expliquez l'architecture en couches et ses responsabilités

Réponse : L'architecture en couches sépare les préoccupations : (1) Controller couche présentation, valide input et appelle services, (2) Service couche logique métier, orchestre opérations et implémente règles, (3) Repository couche persistance, CRUD et requêtes données, (4) Entity

représentation base de données, (5) DTO transfert de données entre couches. Avantages : test unitaire facile (mock Repository), réutilisabilité (service appelé par Controller et autres services), maintenance (change logic impacté qu'une couche). Désavantage : peut être verbeux pour petites applications.

11.2.2 Q2 : Pourquoi utiliser DTOs au lieu de retourner directement des entités ?

Réponse : DTOs découpent l'API interne (ce que le client véritablement consomme) de l'entité persistante. Si on change la structure base de données, le contrat API reste stable. DTOs peuvent aussi contenir différentes données que l'entity (ex : entity a motDePasse qui ne doit jamais être retourné). DTOs optimisent la sérialisation JSON (seulement champs nécessaires) et respectent Single Responsibility. Sans DTO, Entity devient amas de responsabilités (persistance + API + validation), créant des fuites d'abstraction.

11.2.3 Q3 : Quel est le problème N+1 et comment l'identifier/corriger ?

Réponse : Le problème N+1 se produit quand une requête charge N entités, puis pour chacune une requête supplémentaire chargeait données reliées. Exemple : charger 100 utilisateurs (1 query), puis pour chaque utilisateur charger ses commandes (100 queries = 101 total). Identification : logs SQL montrent beaucoup requêtes similaires. Corrections : (1) eager loading avec @Query LEFT JOIN FETCH, (2) projection DTO avec jointure SQL unique, (3) batch loading, (4) caching résultats. Important : minimiser requêtes DB est clé pour performance.

11.2.4 Q4 : Expliquez les niveaux d'isolations transactionnelles

Réponse : (1) READ_UNCOMMITTED : littéralement valeurs non commités (lectures sales), jamais recommandé. (2) READ_COMMITTED (défaut) : lit seulement données commités, mais fantômes possibles. (3) REPEATABLE_READ : même valeur toujours relue même si modifiée par autre transaction, mais nouvelles lignes insérées visibles (fantômes). (4) SERIALIZABLE : isolé complet, comportement équivalent exécution séquentielle. Choix dépend cas d'usage : plupart services web = READ_COMMITTED, transferts montaires = SERIALIZABLE. Trade-off : sécurité vs performances (locks plus stricts = plus lent).

11.2.5 Q5 : Comment gérer les exceptions efficacement en Spring Boot ?

Réponse : Créer exceptions métier personnalisées (RuntimeException étendues, pas checkées). Utiliser @RestControllerAdvice et @ExceptionHandler pour centralize exception handling. Chaque handler retourne ResponseEntity avec HttpStatus et ErrorResponse structuré (timestamp, code, message). Logging : ERROR pour exceptions inattendues, WARN pour erreurs métier prévues. Avantages : réponses consistantes, sécurité (pas exposition détails internes), reusability sans try-catch répété. Important : ne pas reveler stack traces au client en production.

11.2.6 Q6 : Quel est l'avantage de @Transactional et ses pièges ?

Réponse : @Transactional démarre une transaction et gère automatiquement commit/rollback. Avantages : exceptions non-checkées causent rollback, atomicité garanti, code plus clean sans try-catch/rollback manuel. Pièges : (1) exception checkée ne cause pas rollback par défaut (spécifier rollbackFor), (2) appels internes à méthode @Transactional à partir même classe contournent proxy Spring (appeler via self-injection ou déplacer dans autre service), (3) readOnly = true optimise mais provoque exception si mutation, (4) timeout peut causer deadlock. Meilleure pratique : utiliser @Transactional seulement sur services, pas controllers.

11.2.7 Q7 : Comment choisir entre XML et annotations pour configuration Spring ?

Réponse : XML : verbose mais centralisé, visible où beans définis. Annotations : concis et proche code, mais données métadonnées dispersées dans codebase. Consensus moderne : annotations + Java config (@Configuration) pour approche code-first lisible et maintenable. XML reste utile pour legacy projects ou configuration très complexe changeant souvent. Spring Boot préfère application.yml centralisé avec externalisation par profils (dev, prod).

11.2.8 Q8 : Qu'est-ce que la propagation transactionnelle et ses modes ?

Réponse : La propagation détermine si nouvelle transaction créée ou réutilisée. REQUIRED (défaut) : crée nouvelle transaction si aucune, sinon réutilise. REQUIRES_NEW : crée toujours nouvelle, suspend existante. NESTED : nested transaction (rollback partiel possible). Exemple : service A appelle service B avec @Transactional(propagation=REQUIRES_NEW), erreur B ne rollback pas A. REQUIRES_NEW utile quand besoin de logger erreur même si transaction principal fail.

11.2.9 Q9 : Comment configurer et utiliser les profils Spring ?

Réponse : Profils permettent configurations différentes par environnement. Fichiers : application.yml, application-dev.yml, application-prod.yml. Lancer : java -jar app.jar --spring.profiles.active=prod ou dans application.yml : spring.profiles.active=dev. Utilisation code : @Profile("dev") sur @Bean pour création conditionnelle. @ConfigurationProperties (prefix=app) type fort config. Variables environnement : APP_KEY dans yml ou @Value("\${app.key}") Meilleure pratique : sensitive data jamais en prod

11.2.10 Q10 : Expliquez comment Spring injecte les dépendances et ses cycles de vie

Réponse : Spring scanne classpath, crée beans et injecte dépendances via @Autowired (par type, sinon par nom). Cycles : (1) instantiation, (2) property injection, (3) BeanNameAware.setBeanName() si implémenté, (4) ApplicationContextAware.setApplicationContext(), (5) pre-init @PostConstruct, (6) afterPropertiesSet() si InitializingBean. Scopes : singleton (une instance partagée), prototype (nouvelle instance chaque fois), request/session (web). Suppression : @PreDestroy ou destroy(), singleton détruit à arrêt contexte. Important : cycles de vie complexes peuvent cacher bugs, utiliser profils et tests pour valider.

11.3 Questions Kafka, AWS et DevOps

11.3.1 Q1 : Expliquez le mécanisme de partitions dans Kafka et ses bénéfices

Réponse : Chaque topic Kafka est divisé en partitions, chacune stockage ordonné et indépendant. Messages avec même clé vont à même partition, garantissant ordre pour cette clé. Partitions différentes peuvent être lues par consumers différents en parallèle, créant scalabilité horizontale. Exemple : topic avec 3 partitions peut être consommé par jusqu'à 3 consumers simultanément, multipliant throughput par 3. Réplication : chaque partition répliquée sur plusieurs brokers pour haute disponibilité. N de partitions se change pas après création topic (couteux), donc planifier initial avec charge prévue.

11.3.2 Q2 : Qu'est-ce qu'un Consumer Group et comment fonctionne la sémantique de livraison ?

Réponse : Consumer Group ensemble de consumers lisant même topic. Kafka garantit qu'un message est consommé par exactement un consumer du groupe, permettant scalabilité et parallélisme. Sémantiques : (1) At Most Once : message peut être perdu si consumer crash après

récupération mais avant traitement, (2) At Least Once : message peut être dupliqué si consumer crash après traitement mais avant commit offset, (3) Exactly Once : garanti pas duplication/-perte (très strict, overhead). Configuration : `enable.auto.commit=false`, `consumer.commitSync()` après traitement pour At Least Once. Important : Exactly Once requiert idempotence consumer (même message deux fois = résultat identique).

11.3.3 Q3 : Quel est l'offset et comment Kafka track consommation ?

Réponse : Offset entier séquentiel représentant position message dans partition. Consumer group track offset de chaque partition consommation dans topic spécial

11.3.4 Q4 : Décrivez l'architecture AWS et services principaux

Réponse : AWS modèle cloud : utilisateurs paient par usage, pas infrastructure serveur propre. Services principaux : (1) EC2 (machines virtuelles), (2) RDS/DynamoDB (bases données), (3) S3 (stockage objet), (4) Lambda (compute serverless), (5) CloudFront (CDN), (6) ALB/NLB (load balancers), (7) VPC (réseau isolé), (8) IAM (authentification/autorisation). Modèles : IaaS (EC2), PaaS (Elastic Beanstalk), SaaS (services gérés). Avantages : scalabilité, paiement à l'usage, haute disponibilité. Challenges : coûts explosifs si mal config, courbe d'apprentissage.

11.3.5 Q5 : Quel est la différence entre S3, EBS et EFS ?

Réponse : S3 : stockage objet (clé/valeur), illimitée, très durable (99.9999%), cheap, pas système fichiers. EBS : bloc storage (volumes) attaché EC2, haute performance, latence basse, limité taille et une instance EC2. EFS : système fichiers NFS managé, partagé plusieurs instances, scalable automatique, plus cher que EBS. Choix : S3 pour archives/logs/backups, EBS pour performance base de données/système, EFS pour partagé multiple instances. Coûts : S3 cheapest/GB, EFS milieu, EBS cher mais performance.

11.3.6 Q6 : Expliquez Auto Scaling et comment configurer

Réponse : Auto Scaling group crée/supprime instances automatiquement selon métriques. Paramètres : Min capacity (instances minimum), Desired capacity (désired courant), Max capacity (maximum), scaling policies (augment si CPU > 70%, diminue si < 30%). Health checks : vérifie instances saines, remplace les malades automatiquement. Important : définir Launch Template (AMI, instance type, user data). Avantages : aucune intervention manuelle, handle pics charge, cost efficient (scale down la nuit). Challenges : warmup time (instances neuves prennent temps démarrer), métriques may lag réalité.

11.3.7 Q7 : Qu'est-ce qu'un IAM Role et pourquoi c'est important ?

Réponse : IAM Role ensemble de permissions qu'instance EC2 assume, permettant accéder autres services AWS (S3, RDS, etc.) sans hardcoding credentials. Best practice : jamais mettre AWS keys en code, utiliser IAM roles. Création : définir policy JSON granté (ex : `s3:GetObject` sur bucket spécifique), attacher à instance. Instance automatiquement obtient credentials temporaires via metadata service. Avantages : sécurité (pas exposition keys), rotation automatique, audit trail. Important : least privilege (permissions seulement nécessaires), jamais * sur ressources sensibles (ex : pas `s3:*` sur tous buckets).

11.3.8 Q8 : Comment optimiser coûts AWS ?

Réponse : Stratégies : (1) Right-sizing : instances trop larges = gaspillage, utiliser monitoring trouver petit assez, (2) Reserved Instances : 30-70% réduction pour 1-3 ans engagement, bien

si charge prévisible, (3) Spot Instances : 70-90% réduction mais peut être terminée AWS, bon pour batch jobs, (4) Shutdown dev environments : env dev seulement pendant travail (scripts schedule), (5) Data transfer : éviter cross-region très cher, (6) S3 Intelligent-Tiering : auto move données peu accédées à Glacier cheaper, (7) Consolidate workloads : mettre plusieurs petites apps sur une instance, pas une par app. Monitoring : CloudWatch logs + AWS Cost Explorer identifier culprits.

11.3.9 Q9 : Expliquez une CI/CD pipeline type

Réponse : Pipeline stages : (1) Source : commit code, webhook déclenche, (2) Build : compile, dependency resolution, produce artifact, (3) Test : unit tests, integration tests, 95%+ couverture idéal, (4) Deploy staging : déploie version candidate pour validation, (5) Approval : humain review ou automatique, (6) Deploy production : livraison finale. Tooling : GitLab CI/CD, GitHub Actions, Jenkins. Best practices : fail fast (arrêter si test fail), immutable artifacts (build une fois, déploie partout), secrets management (pas en git). Monitoring post-deploy : health checks, logs, rollback plan si failure.

11.3.10 Q10 : Qu'est-ce qu'une Infrastructure as Code et pourquoi l'utiliser ?

Réponse : Infrastructure as Code (IaC) définit infrastructure en fichiers texte (Terraform, CloudFormation, Ansible), versionné en git. Avantages : reproductibilité (même config every time), tracking changes (git history), automatisation (destroy/recreate environments), disaster recovery (redéploi rapidement). Terraform : agnostique cloud (AWS, Azure, GCP), langage HCL lisible. CloudFormation : AWS-specific, JSON/YAML. Workflows : define resources, plan (preview changes), apply (crée). Important : secrets (DB passwords) jamais en repo, utiliser HashiCorp Vault ou AWS Secrets Manager.

11.4 Questions Frontend Angular et Patterns Avancés

11.4.1 Q1 : Expliquez la différence entre Observable et Promise

Réponse : Promise retourne une valeur unique, immuable une fois créée, non annulable. Observable peut émettre zéro, une, ou plusieurs valeurs sur le temps, annulable via unsubscribe, lazy (exécuté seulement subscribe). Promises : utile une opération async (fetch), async/await syntaxe concise. Observables : utile streams de données (WebSocket, events). RxJS operators (map, filter, switchMap) rendent Observable puissant. Cependant : Observable requiert subscription, sinon ne s'exécute jamais, contrairement Promise qui exécute immédiatement. Memory leak si oublie unsubscribe, utiliser async pipe (auto-unsubscribe) ou takeUntil(destroy\$).

11.4.2 Q2 : Qu'est-ce que Change Detection et comment optimiser ?

Réponse : Change Detection processus Angular vérifie si données composant changées et update DOM. Par défaut CheckAlways (vérifie chaque événement). Optimisation : ChangeDetectionStrategy.OnPush (vérifie seulement si @Input change ou event du composant). Si utilise Observables, async pipe déclenche changement quand Observable émet. markForCheck() manuellement si besoin. Performance : OnPush peut gagner 5-10x sur composants complexes. Important : @Input doivent être immutables avec OnPush, sinon Angular manque changements (pass objets nouveaux au lieu de modifier).

11.4.3 Q3 : Expliquez RxJS switchMap vs mergeMap vs concatMap

Réponse : switchMap : annule Observable précédent, lance nouveau. Utile search auto-complete (utilisateur change search term, ancien request annulé). mergeMap : lance nouveau sans annuler, résultats souscrits parallèle. Utile quand tous résultats pertinents (multi upload).

concatMap : attend Observable précédent complet avant lancer nouveau. Utile ordre important (queue de tâches). Exemple : search box = switchMap (utilisateur metire plus loin), file upload = concatMap (resp. ordre).

11.4.4 Q4 : Comment gérer erreurs et retry en RxJS ?

Réponse : catchError operator : capture erreur et retourner Observable fallback. retry(3) : réessai automatique 3 fois avant erreur. retryWhen : plus contrôle sur retry logic (ex : exponential backoff). Exemple : `“.userService.getUser(id).pipe(retry(3),catchError(err => logger.error(err));return of(defaultUser); // Fallback)).subscribe(user => this.user = user);“` : ordre de pérateurs compte (retry après source, catchError dernier).

11.4.5 Q5 : Quel est le cycle de vie des composants Angular ?

Réponse : (1) constructor : création instance, injections, pas données @Input. (2) ngOnChanges : @Input changé, appelé avant ngOnInit et à chaque changement. (3) ngOnInit : première initialisation, API calls. (4) ngDoCheck : détection changement custom. (5) ngAfterContentInit : projections de contenu initialisées. (6) ngAfterViewInit : template et enfants rendus, éviter modifiant inputs. (7) ngOnDestroy : nettoyage subscriptions, timers, avant destruction. Important : utiliser ngOnInit pour logic initialisation, jamais constructor. ngOnDestroy obligatoire pour unsubscribe.

11.4.6 Q6 : Expliquez Guards (CanActivate, CanDeactivate)

Réponse : CanActivate : protéger route, vérifier authentication/autorisation. CanDeactivate : confirmer avant quitter route (ex : form non sauvegardé). Implémentation : classe implémente interface, retourne boolean ou Observable<boolean>. Utilisation routing : `“routes = [ path : 'admin', component : AdminComponent, canActivate : [AuthGuard], data : role : 'ADMIN' , path : 'user', component : UserComponent, canDeactivate : [ConfirmDeactivateGuard] ] ;“`@Injectable() AuthGuard implémente CanActivate, retourne isAuthenticated() observable. Important : ordre guards compte, multiplesGuards tous doivent passer.

11.4.7 Q7 : Comment optimiser performance Angular app ?

Réponse : Stratégies : (1) Lazy loading : routes chargées à la demande au lieu d'au démarrage, réduit bundle initial, (2) ChangeDetectionStrategy.OnPush : moins de vérification, (3) trackBy avec *ngFor : réutilise instances DOM au lieu recréer, (4) Unsubscribe subscriptions : éviter memory leaks, (5) Virtual scrolling : affiche seulement visible items pour grandes listes, (6) Code splitting : séparer en petits bundles, (7) Preloading strategy : charger critical routes en arrière plan, (8) Image optimization : WebP, lazy loading images. Monitoring : lighthouse, webpack-bundle-analyzer pour identify bottlenecks.

11.4.8 Q8 : Qu'est-ce que Dependency Injection et pourquoi c'est crucial ?

Réponse : DI injecte dépendances (services) dans composant au lieu de créer directement. Avantages : découplage (composant ne connaît pas création service), test facile (mock service), réutilisabilité. Dans Angular : @Injectable() décore service, inject dans constructor. Scopes : providedIn : 'root' (singleton tout app), providedIn : 'component' (une instance par composant). Importer dans module ou lazy-loaded component. Anti-pattern : new ServiceClass() au lieu d'inject, perd DI benefits.

11.4.9 Q9 : Expliquez Signals et Effects (Angular 17+)

Réponse : Signals réactivité légère : `count = signal(0)`, update via `count.set(5)` ou `count.update(c => c+1)`, template automatiquement mis à jour. `computed()` crée signal dérivé (`doubl = computed(() => count()*2)`). `effect()` exécute code quand signal change (side effects). Plus simple que RxJS pour state local, pas besoin subscription. `combineLastSignals()` pour combiner. Important : signals remplacent pas RxJS, complémentaires (Signals = local state, RxJS = async data).

11.4.10 Q10 : Qu'est-ce que State Management et quand en avoir besoin ?

Réponse : State management centralise application data, évitant prop drilling (passing inputs travers composants). Libraries : NgRx (Redux pattern), Akita, MobX. Quand éviter : petite app (< 10 composants), simple state. Quand utiliser : app complexe, même data accédée partout, undo/redo, time-travel debugging utile. NgRx architecture : Store (source vérité), Selectors (requêtes), Actions (events), Reducers (pure functions transform state). Important : complexity tax, 20% app overhead, petite app overkill. Start simple (BehaviorSubject service), upgrade si besoin.

Conseils d'Entretien

- **Toujours expliquer le pourquoi** : pas juste la syntaxe, mais la raison architecturale
- **Donner des exemples réels** : montrer compréhension pratique
- **Discuter trade-offs** : avantages, limites, cas d'usage
- **Montrer curiosité** : demander clarifications, explorer alternatives
- **Code propre** : expliquer philosophie de design
- **Reconnaître les gaps** : honête quand vous ne savez pas, mais montrez volonté d'apprendre